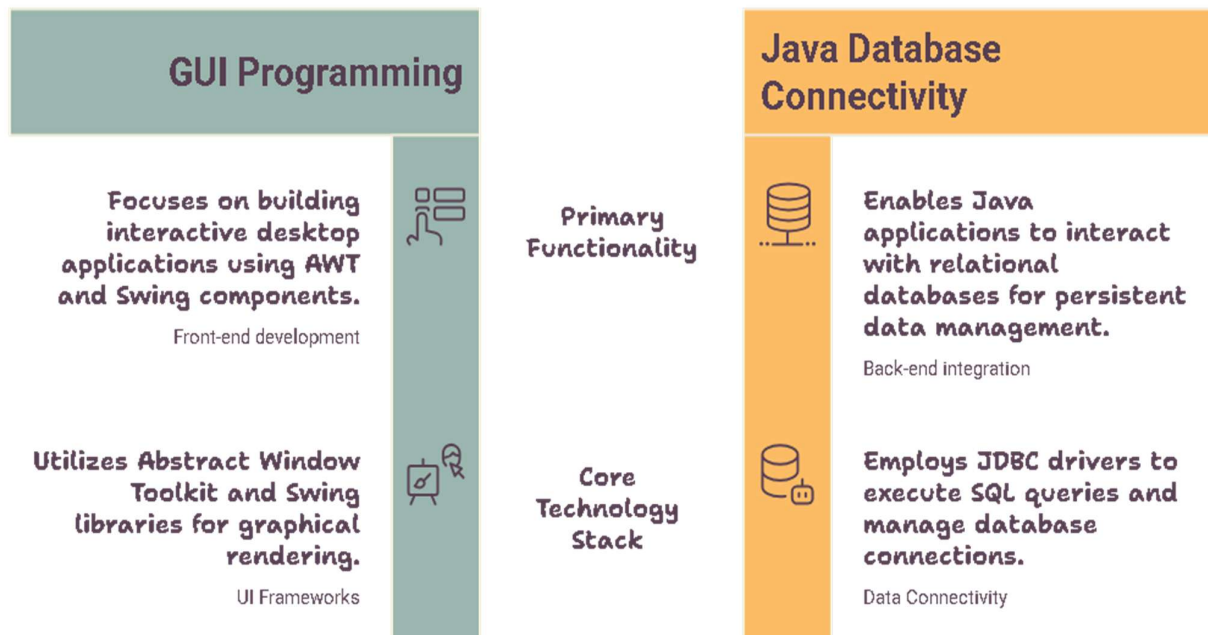


Group D: GUI Programming(awt, swing) & Java Database Connectivity

Experiment No.11

1. **Title:** Demonstrate the AWT Components with Retail Cost Calculator.

2. **Problem Statement:**

Create a GUI application where the user enters the wholesale cost of an item and its markup percentage into text fields. (For example, if an item's wholesale cost is ₹5 and its markup percentage is 100%, then its retail price is ₹10.) The application should have a button that displays the item's retail price when clicked.

3. **Objectives:**

- To design a simple GUI using Java Swing.
- To accept wholesale cost and markup percentage as input.
- To calculate the retail price of an item.
- To handle button click events using ActionListener.

4. **Outcomes:**

Students will be able to learn and understand

- Create basic Java GUI applications & Process user inputs and perform calculations.
- Implement event-driven programming.
- Display results through a graphical interface.
- Develop interactive desktop applications using Java Swing.

5. **Software/Hardware/Other Requirements:**

- Any CPU with Pentium Processor / similar, 256 MB RAM/ more, 1GB HDD / more.
- Operating System – ubuntu/Fedora 64bit OS
- Software: JDK 17 or later, Code Editor

6. **Theory:**

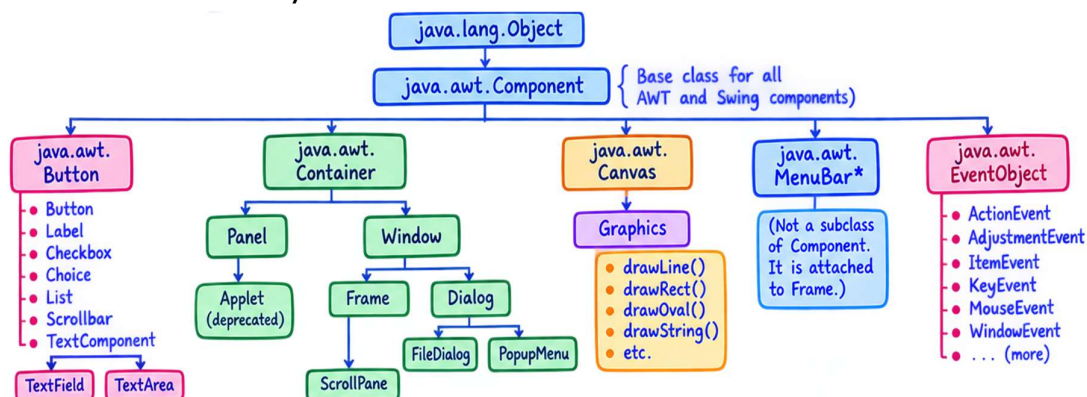
A. Java AWT (Abstract Window Toolkit)

- AWT (Abstract Window Toolkit) is the earliest GUI toolkit provided by Java for developing window-based applications.
- It is a part of the java.awt package and provides components such as buttons, labels, text fields, checkboxes, menus, and containers to create graphical user interfaces.
- AWT components are platform-dependent because they use the native GUI resources of the operating system, making them heavyweight components.

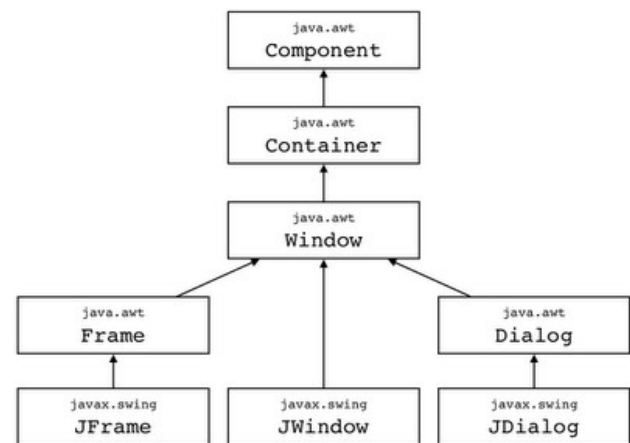
• Features of AWT

- 1) Provides GUI components like Button, Label, TextField, Checkbox, List, and Menu.
- 2) Supports event handling through listeners.
- 3) Uses layout managers such as FlowLayout, BorderLayout, and GridLayout.
- 4) Platform-dependent and heavyweight.
- 5) Forms the foundation on which Swing is built.

• AWT Class Hierarchy

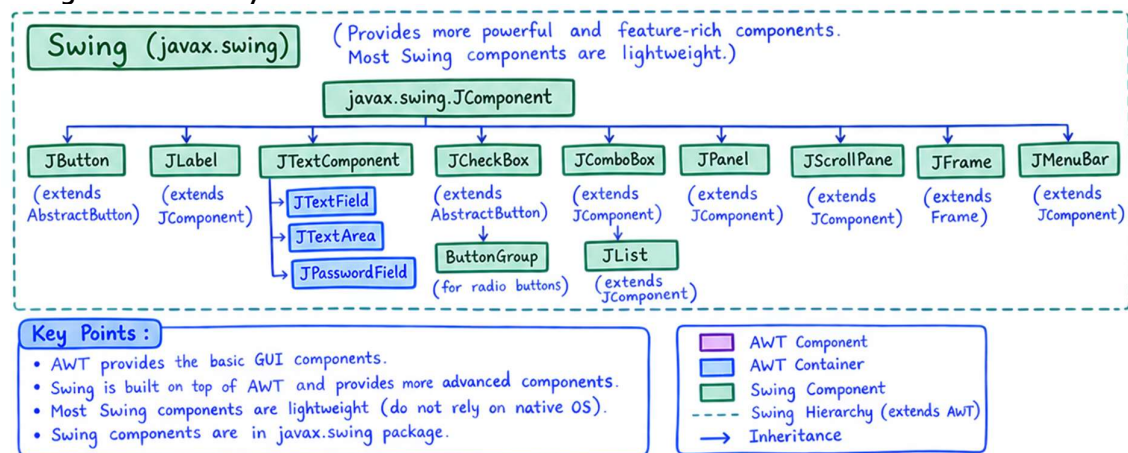


- The hierarchy shows that Component is the base class for all GUI elements, while Container is a special component that can hold other components. Classes such as Frame, Dialog, Panel, and Applet are derived from Container.



B. Java Swing

- Swing** is a part of **Java Foundation Classes (JFC)** used to create **Graphical User Interface (GUI)** applications such as desktop windows, forms, calculators, and management systems.
- It is available in the **javax.swing** package and provides a rich set of lightweight, platform-independent components for building interactive applications.
- Swing Class Hierarchy**



- Advantages of Swing**
 - 1) Easy to develop user-friendly desktop applications.
 - 2) Large collection of reusable GUI components.
 - 3) Supports MVC architecture.
 - 4) Provides consistent appearance across different operating systems.
- AWT vs Swing**

Feature	AWT	Swing
Package	java.awt	javax.swing
Components	Heavyweight	Lightweight
Look and Feel	Platform-dependent	Platform-independent
Customization	Limited	Highly customizable
Performance	Uses native controls	Pure Java implementation
Modern Usage	Limited	More widely used

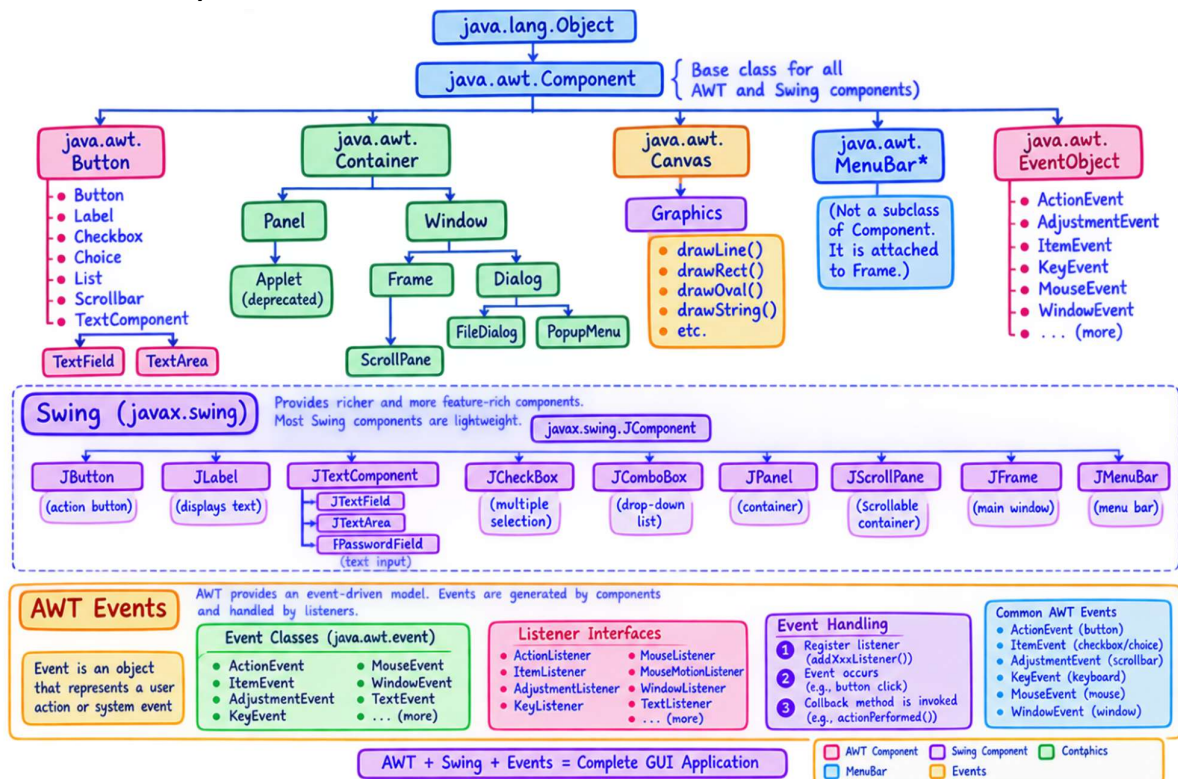
C. AWT Events

- AWT Event Handling** is a mechanism by which a program responds to **user actions** such as button clicks, key presses, mouse movements, or window operations.
- Java AWT follows the **Delegation Event Model**, where an event source generates an event and a listener handles it
- Components of Event Handling**
 - 1) **Event Source** – Object that generates an event (Button, TextField, CheckBox, etc.).
 - 2) **Event Object** – Contains information about the event.
 - 3) **Event Listener** – Interface that receives and processes the event.

Common AWT Events and Listeners

Event Class	Listener Interface	Description
ActionEvent	ActionListener	Generated when a button is clicked
ItemEvent	ItemListener	Generated when an item is selected/deselected
KeyEvent	KeyListener	Generated when a key is pressed, released, or typed
MouseEvent	MouseListener	Generated by mouse actions
MouseMotionEvent	MouseMotionListener	Generated when mouse is moved or dragged
WindowEvent	WindowListener	Generated when window is opened, closed, etc.
FocusEvent	FocusListener	Generated when a component gains or loses focus
TextEvent	TextListener	Generated when text changes in a text component

D. Summary of all three



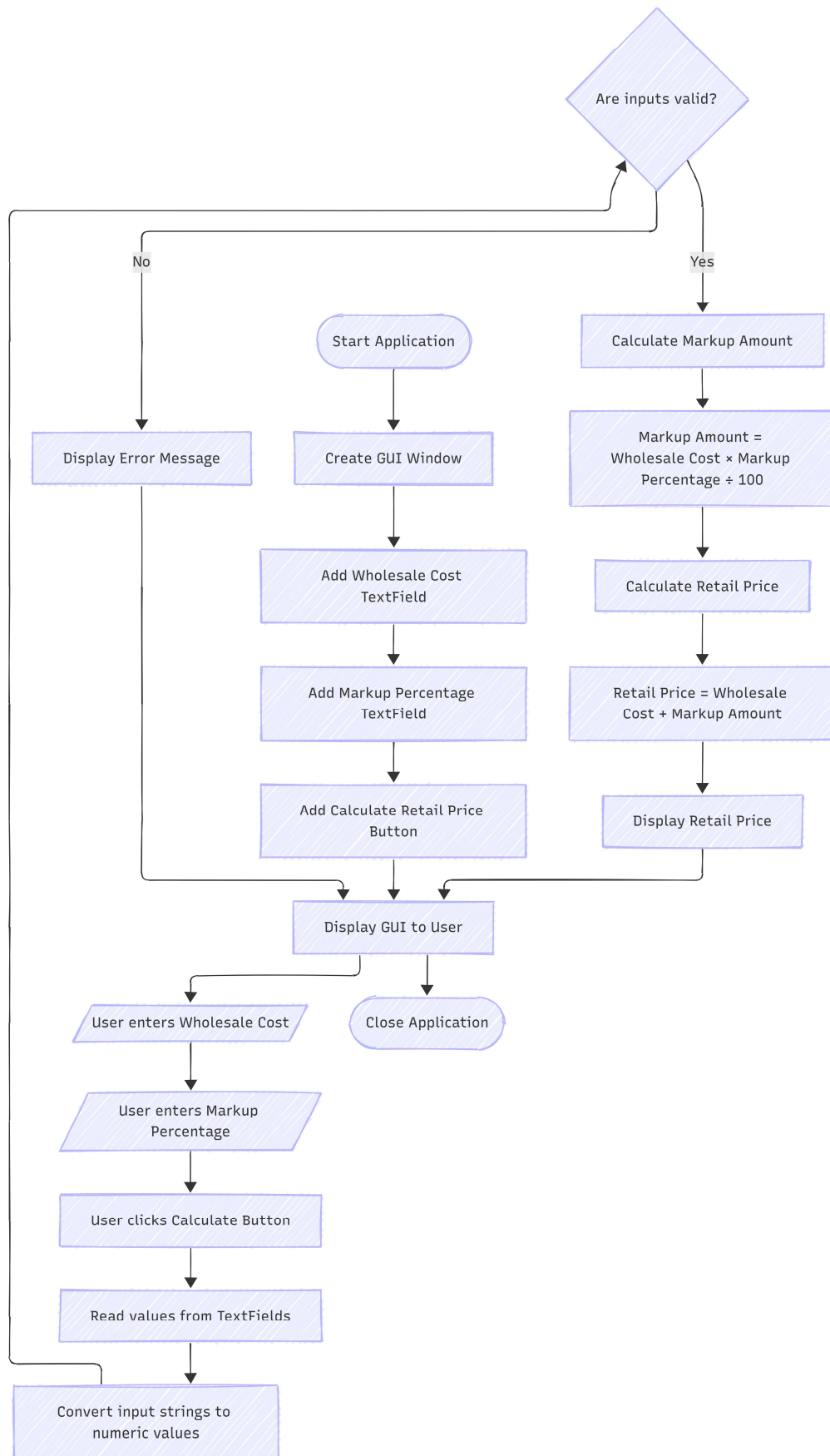
E. Short Description of problem Statement:

- In this application:
 - The user enters the wholesale cost and markup percentage in text fields.
 - When the Calculate button is clicked, an event is generated and handled using ActionListener.
 - The retail price is calculated using the formula:

$$\text{Retail Price} = \text{Wholesale Cost} + \left(\text{Wholesale Cost} \times \frac{\text{Markup Percentage}}{100} \right)$$

- The calculated retail price is then displayed on the GUI.
- Swing Components Used:**
 - JFrame**: Creates the application window.
 - JLabel**: Displays text labels.
 - JTextField**: Accepts user input.
 - JButton**: Triggers the calculation.
 - ActionListener**: Handles button click events.
- This application demonstrates the concepts of GUI programming, event handling, user input processing, and arithmetic calculations in Java.
- JFrame** provides the main application window, **JLabel** displays text information, **JTextField** accepts user input, **JButton** performs actions when clicked, and **ActionListener** handles button-click events. Together, these components are used to create interactive Java Swing GUI applications.

7. Flowchart Diagram:

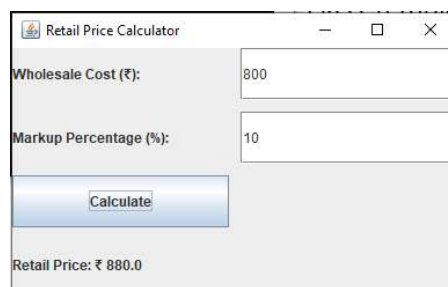


8. Implementation:

Program for implementation of Retail Price Calculation

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class RetailPriceCalculator extends JFrame implements ActionListener {
    private JLabel lblWholesale, lblMarkup, lblResult;
    private JTextField txtWholesale, txtMarkup;
    private JButton btnCalculate;
    public RetailPriceCalculator() {
        setTitle("Retail Price Calculator");
        setSize(400, 250);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        // Create Components
        lblWholesale = new JLabel("Wholesale Cost (₹):");
        lblMarkup = new JLabel("Markup Percentage (%):");
        lblResult = new JLabel("Retail Price: ");
        txtWholesale = new JTextField(15);
        txtMarkup = new JTextField(15);
        btnCalculate = new JButton("Calculate");
        // Register Event
        btnCalculate.addActionListener(this);
        // Layout
        setLayout(new GridLayout(4, 2, 10, 10));
        add(lblWholesale);
        add(txtWholesale);
        add(lblMarkup);
        add(txtMarkup);
        add(btnCalculate);
        add(new JLabel());
        add(lblResult);
        setVisible(true);
    }
    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            double wholesaleCost = Double.parseDouble(txtWholesale.getText());
            double markupPercentage = Double.parseDouble(txtMarkup.getText());
            if (wholesaleCost < 0 || markupPercentage < 0) {
                JOptionPane.showMessageDialog(this, "Values cannot be negative!",
                    "Input Error", JOptionPane.ERROR_MESSAGE);
                return;
            }
            double retailPrice = wholesaleCost + (wholesaleCost * markupPercentage / 100);
            lblResult.setText("Retail Price: ₹ " + retailPrice);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this,
                "Please enter valid numeric values.", "Input Error",
                JOptionPane.ERROR_MESSAGE);
        }
    }
    public static void main(String[] args) {
        new RetailPriceCalculator();
    }
}
```

Output:



9. Conclusion:

This GUI application demonstrates how Java Swing components and event handling can be used to create an interactive retail price calculator. The user enters the wholesale cost and markup percentage, and the application calculates and displays the final retail price when the button is clicked.

10. Review Questions & Exercises:**a. Fill in the Blanks**

Q. No.	Question	BT Level	CO	PO & PSO
i	Java Swing applications are commonly created using a _____ as the main window.	BT1	CO3	PO1, PSO1
ii	Invalid user input can be handled using _____ handling.	BT1	CO3	PO1, PSO1
iii	The getText() method is used to _____ text from a JTextField.	BT1	CO3	PO1, PSO1

b. Answer the following:

Q. No.	Question	BT Level	CO	PO & PSO
i	Explain different AWT Layouts for arranging components.	BT2	CO3	PO1, 2, PSO1
ii	Explain JFrame in detail with methods, constructor and example.	BT2	CO3	PO1, 2, PSO1
iii	Describe the process of converting user input into numerical values.	BT3	CO3	PO1, 2,5, PSO1

c. Programming Assignments:

Q. No.	Question	BT Level	CO	PO & PSO
i	Shopping Cart Application: Create a Java Swing GUI application that simulates an online shopping system where users can select multiple products, enter the desired quantity for each product, and generate a detailed bill. The application should calculate the subtotal based on selected items, apply discounts according to predefined purchase slabs, compute GST on the discounted amount, and display the final payable amount. The generated bill should include product-wise details, quantity, unit price, subtotal, discount, tax, and net amount, providing users with a complete shopping invoice.	BT3	CO3	PO1,3,5, PSO1
ii	Loan Eligibility and EMI Calculator: Create a Java Swing GUI application that assists users in evaluating loan affordability and repayment details. The application should accept the applicant's monthly salary, loan amount, annual interest rate, and loan tenure as input. Upon clicking the Calculate button, it should compute the Equated Monthly Installment (EMI), total repayment amount, and total interest payable. Based on predefined eligibility criteria such as salary, EMI-to-income ratio, and requested loan amount, the application should determine whether the applicant qualifies for the loan and display the approval or rejection status along with the calculated financial details.	BT4	CO3	PO1, 2,3, 5, PSO1

11. References:

- Java 6 Programming, Black Book, Dreamtech
- Paul Deitel and Harvey Detail, Java: How to Program, Pearson's Publication, 9th Edition
- Herbert Schildt, (2021). Java: The complete reference, 13th edition. McGraw-Hill Education

Performance Indicators	
Process related (6 Marks)	
1	Logic formation (3)
2	Debugging ability (2)
3	Follow ethical practices (1)
Product related (4 Marks)	
4	Expected output (1)
5	Timely Submission (2)
6	Answer to sample questions (1)

Marks Obtained			Dated signature of Teacher
Process Related (6)	Product Related (4)	Total (10)	